



DATA-DRIVEN INSIGHTS AND PREDICTIVE MODELING IN THE USED CAR MARKET

Artem Aloian
Chin-Chung Wang
Ahishek Subramani
KuanMing Su

WELCOME TO OUR PREDICTIVE ANALYTICS

The automotive industry is a global economic powerhouse, generating billions annually and providing employment for millions. The used car market stands out due to its affordability and alignment with changing consumer preferences.

This market presents unique opportunities for predictive analytics, as vehicle pricing is influenced by a complex interplay of factors, including market demand, economic conditions, and vehicle-specific attributes.



This project utilizes a comprehensive dataset to develop predictive models, aiming to optimize pricing strategies and inventory management for stakeholders like dealerships, consumers, and manufacturers.

Dataset Selection

```
[ ] print("Size of the dataset:", df.shape)
    print(f'Number of rows: {df.shape[0]}, number of columns: {df.shape[1]}')
```

```
⇒ Size of the dataset: (558837, 16)
   Number of rows: 558837, number of columns: 16
```

```
[ ] df.drop(['vin', 'model'], axis=1, inplace=True)
```

The reason why we drop these columns:

vin: It is vehicle ID, we do need to use it for our predictive model

model: model & body present the same information what category is this car. Each brand iust using differnet model name to describe therefor we will drop (model) column from our data. (Body) has duplication for example SUV & suv.

```
[ ] numeric_columns = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
    print("Numeric columns:", numeric_columns)

    categorical_columns = df.select_dtypes(include=['object']).columns.tolist()
    print("Category columns:", categorical_columns)
```

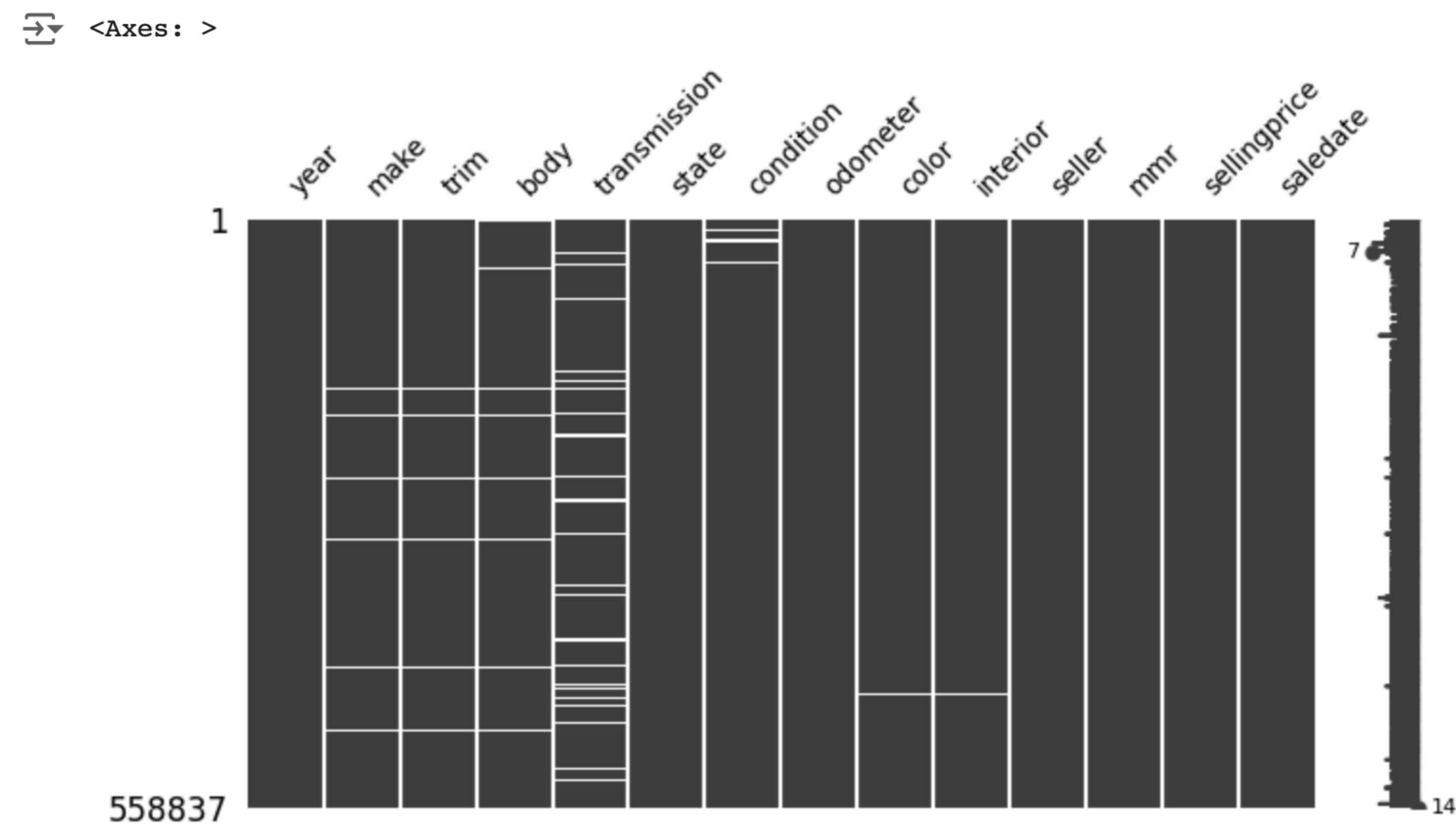
```
⇒ Numeric columns: ['year', 'condition', 'odometer', 'mmr', 'sellingprice']
   Category columns: ['make', 'trim', 'body', 'transmission', 'state', 'color', 'interior', 'seller', 'saledate']
```



	Missing Values	Percentage
transmission	65352	11.694287
body	13195	2.361154
condition	11820	2.115107
trim	10651	1.905922
make	10301	1.843292
color	749	0.134028
interior	749	0.134028
odometer	94	0.016821
mmr	38	0.006800
sellingprice	12	0.002147
saledate	12	0.002147

Missing value

```
msno.matrix(df, figsize=(8, 4), fontsize=10)
```



We start cleaning data from bottom

color, interior, odometer, mmr, sellingprice, saledate: Low Percentage of Missing Values < 1%

Approach: Listwise Deletion: Removing entire rows where any data is missing. Simple but can lead to significant data loss.

```
▶ # Drop rows where any of the specified columns have missing values
df.dropna(subset=['color', 'odometer', 'mmr', 'sellingprice', 'saledate'], inplace=True)
```

body, trim, make: Create a dictionary to store the mode of each column

```
[ ] value = {}
    for col in ['body', 'trim', 'make']:
        value[col] = df[col].mode().iloc[0] # Store the mode for each column

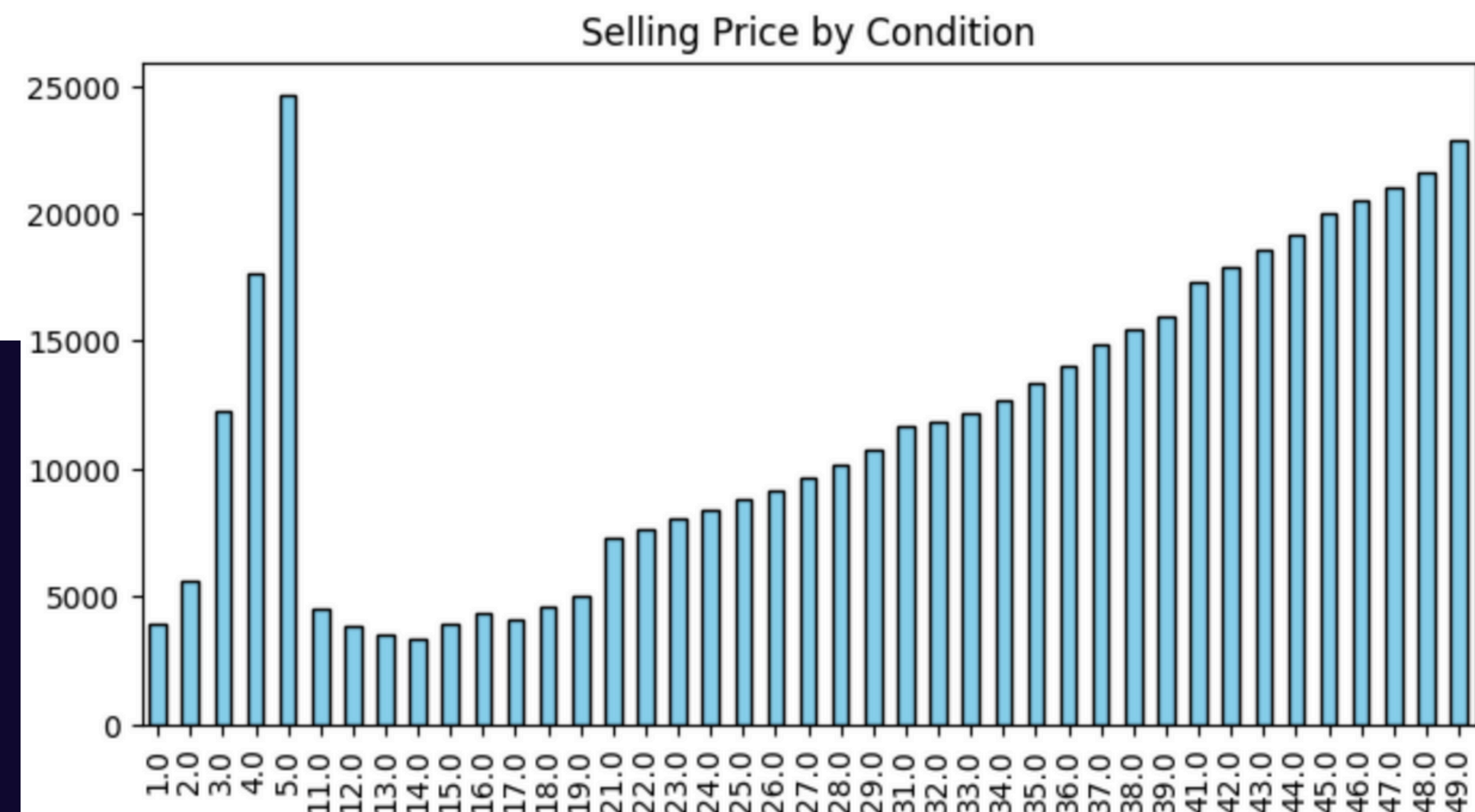
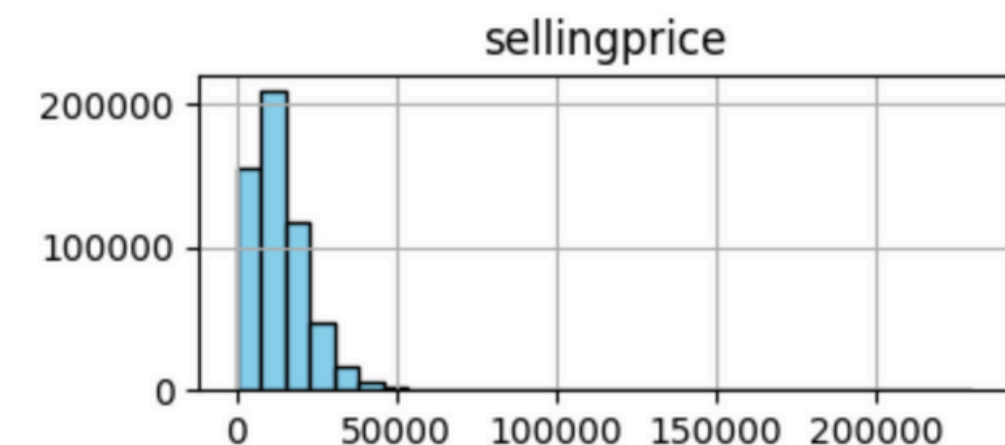
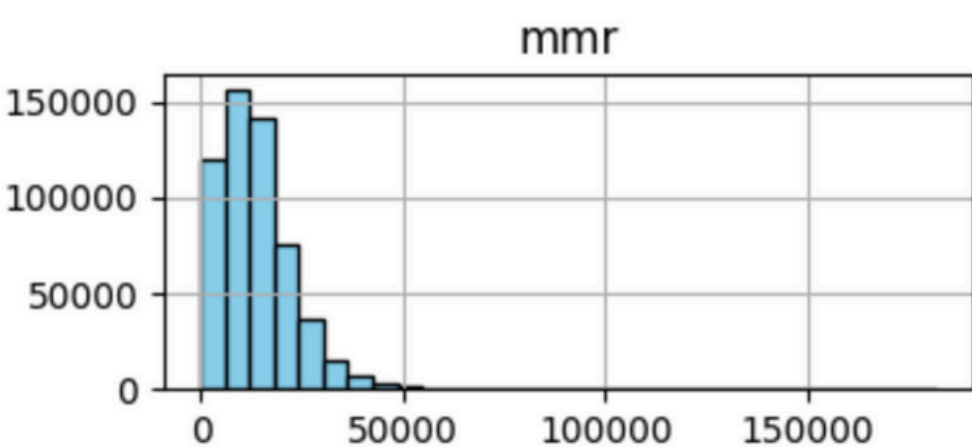
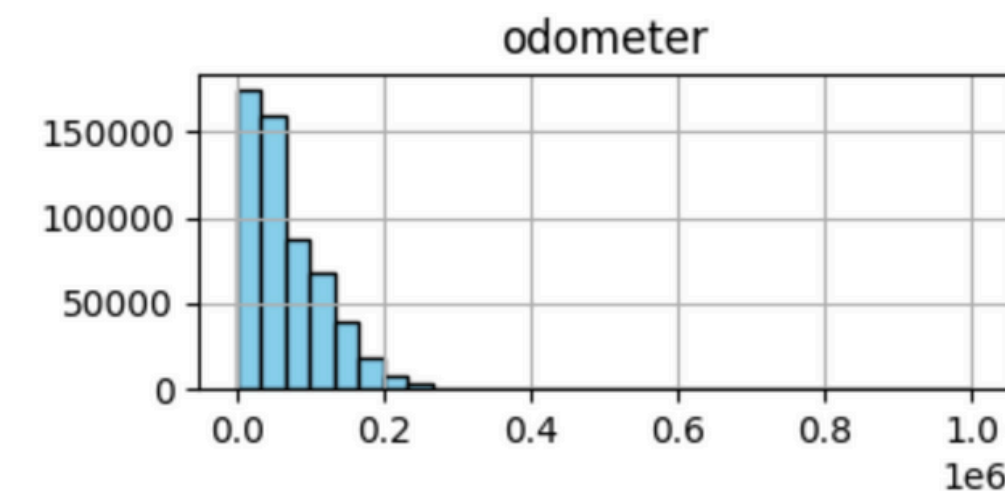
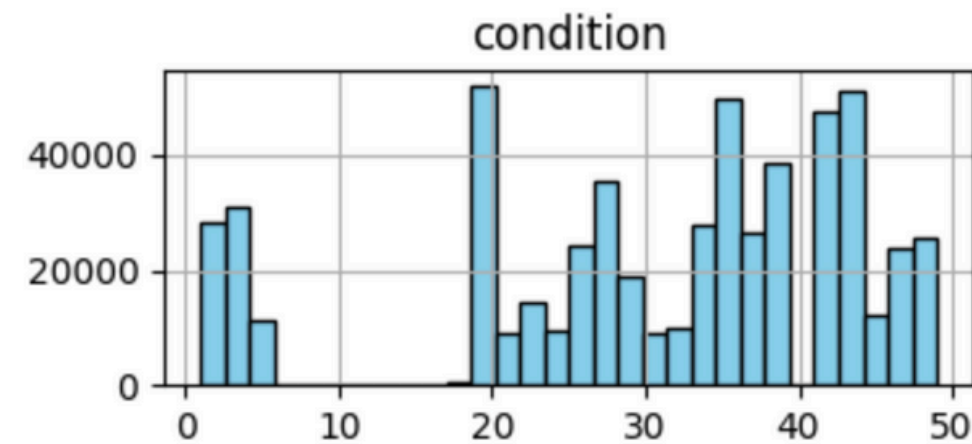
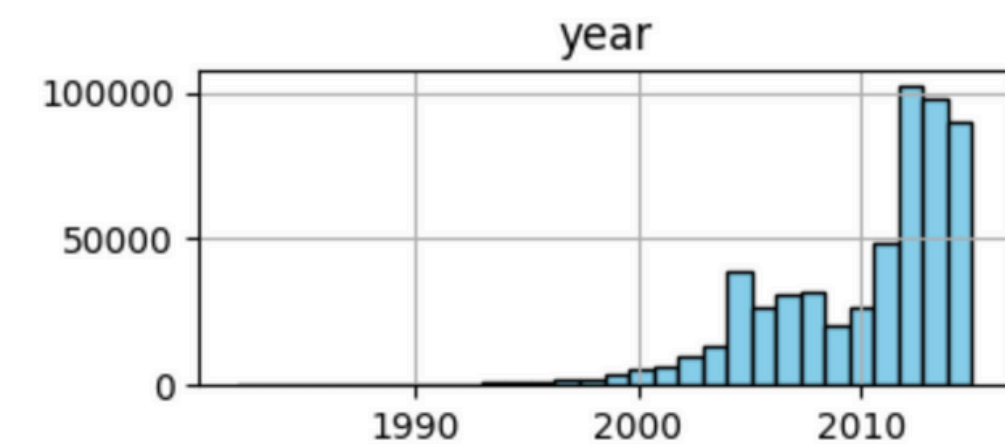
# Fill missing values in categorical columns with their respective mode
df.fillna(value=value, inplace=True)
```

condition have positive relationship with (year), so we use average condition score by year to fill na value.

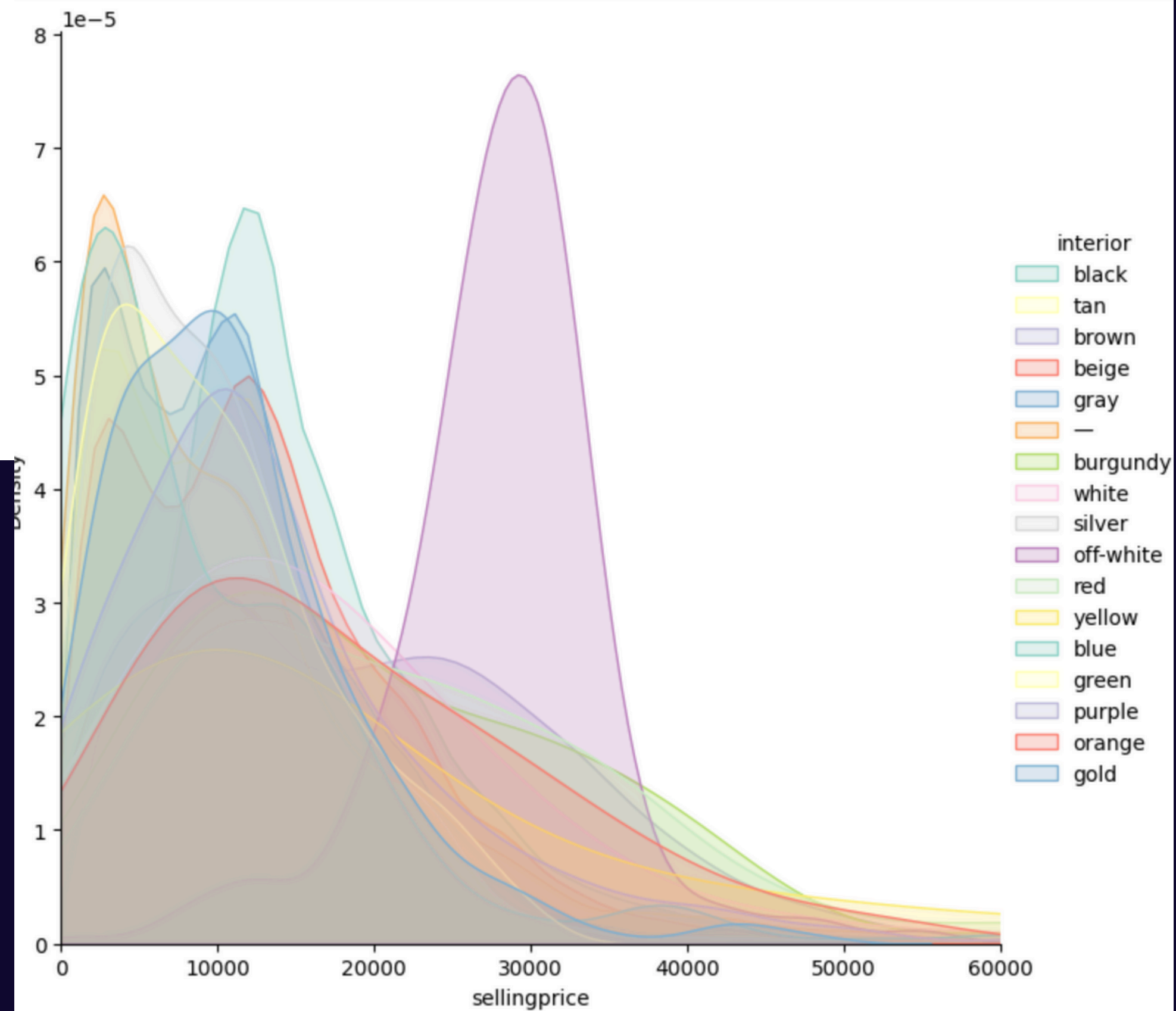
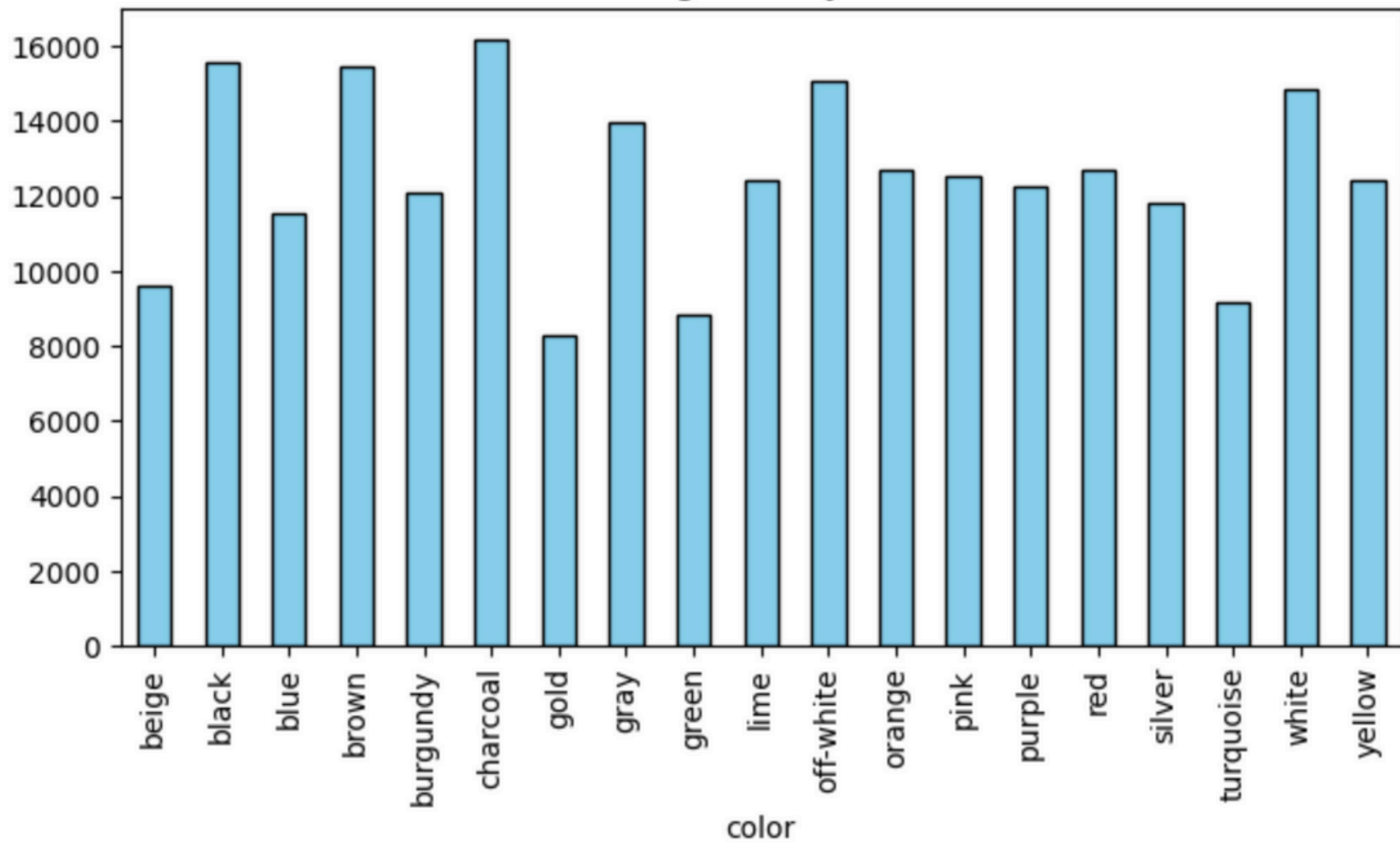
```
[ ] # Define a function to get the mode for each group
def fill_with_group_mode(series):
    return series.fillna(series.mode()[0])

# Group by 'year' and apply the mode-filling function to the 'condition' column
df['condition'] = df.groupby('year')['condition'].transform(fill_with_group_mode)
```

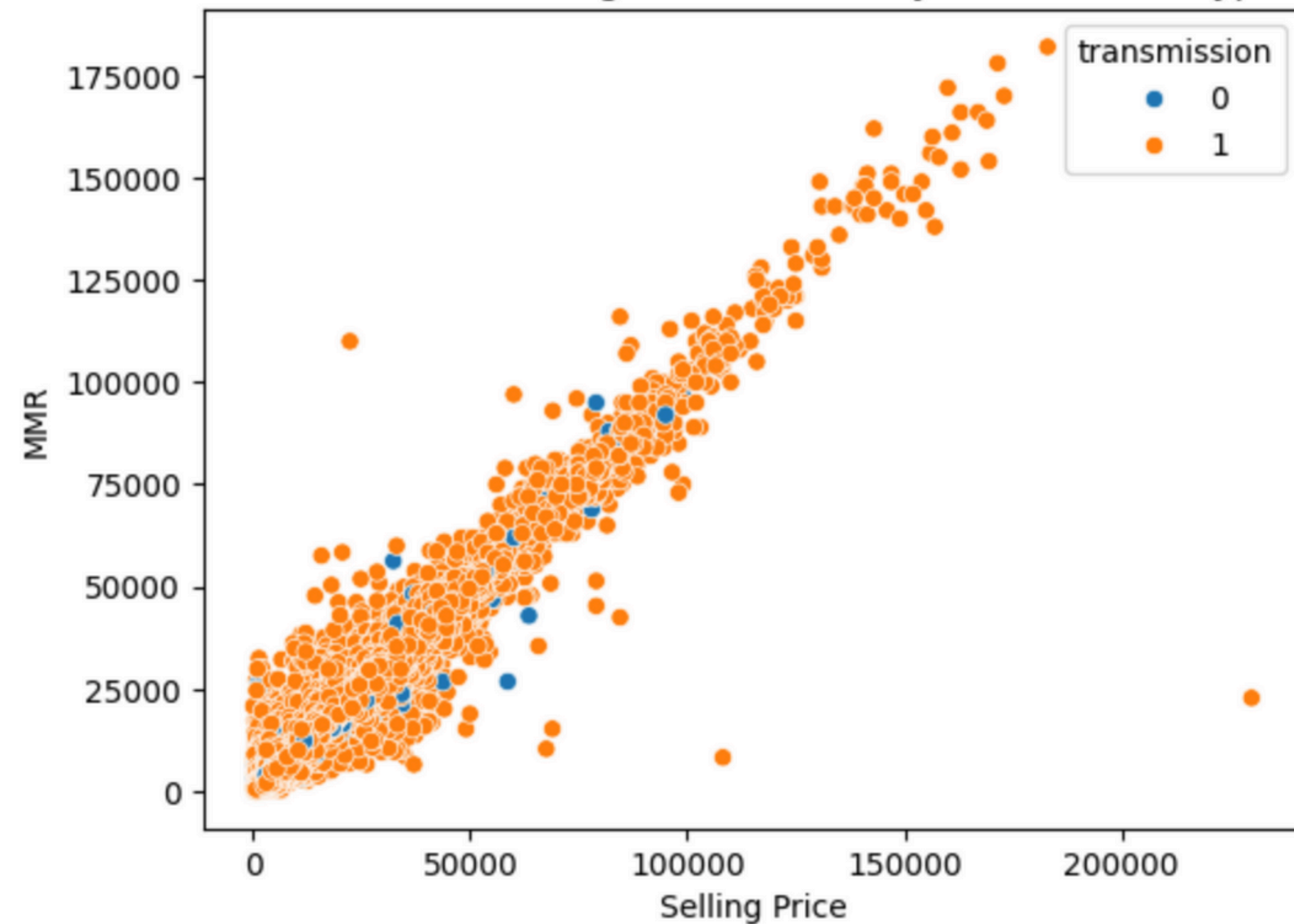
Exploratory data analysis



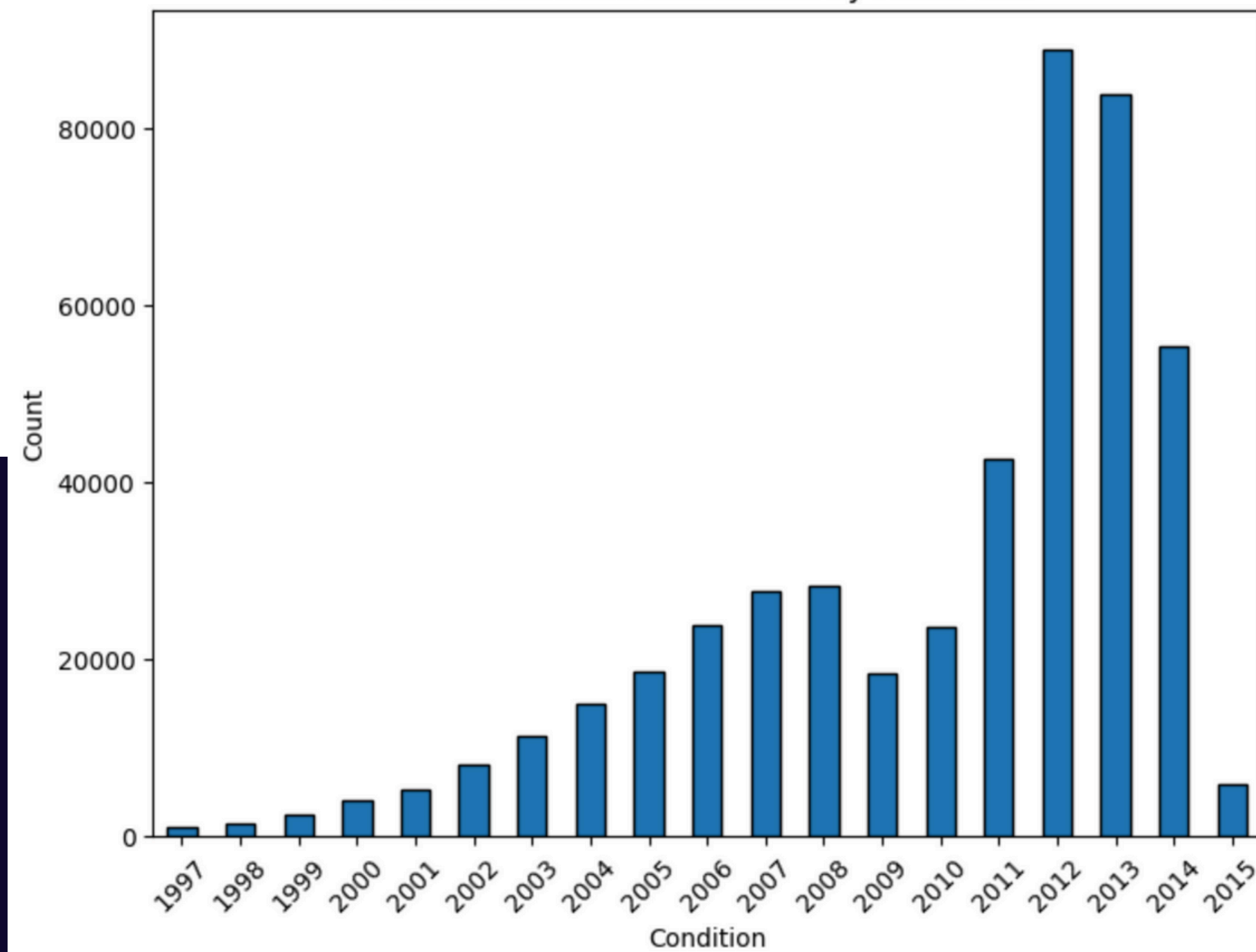
Selling Price by color



Scatter Plot of Selling Price vs MMR by Transmission Type



Vehicle Condition Distribution by Model Year



3. Data Cleaning and Preprocessing

Outlier

```
[ ] import seaborn as sns
```

```
# Define the columns you want to plot, which are numerical and meaningful for outliers detection
numerical_columns = ['year', 'condition', 'odometer', 'mmr', 'sellingprice']
```

```
# Set the aesthetic style of the plots (optional)
sns.set_style("whitegrid") # Other options include: darkgrid, white, dark, ticks
sns.set_context("notebook") # Other options include: paper, notebook, talk, poster
sns.set_palette("pastel") # Other options include: deep, muted, bright, dark, colorblind
```

```
# Create a figure and a set of subplots with a 3x2 grid (sufficient for 5 plots)
fig, axes = plt.subplots(3, 2, figsize=(12, 15))
```

```
# Define flier properties to make outliers red
flierprops = flierprops=dict(marker='o', markeredgecolor='red', alpha=0.5)
```

```
sns.boxplot(x=df['year'], ax=axes[0, 0], whis=3, flierprops=flierprops)
axes[0, 0].set_title('Box Plot of model year')
```

```
sns.boxplot(x=df['condition'], ax=axes[0, 1], whis=3, flierprops=flierprops)
axes[0, 1].set_title('Box Plot of condition')
```

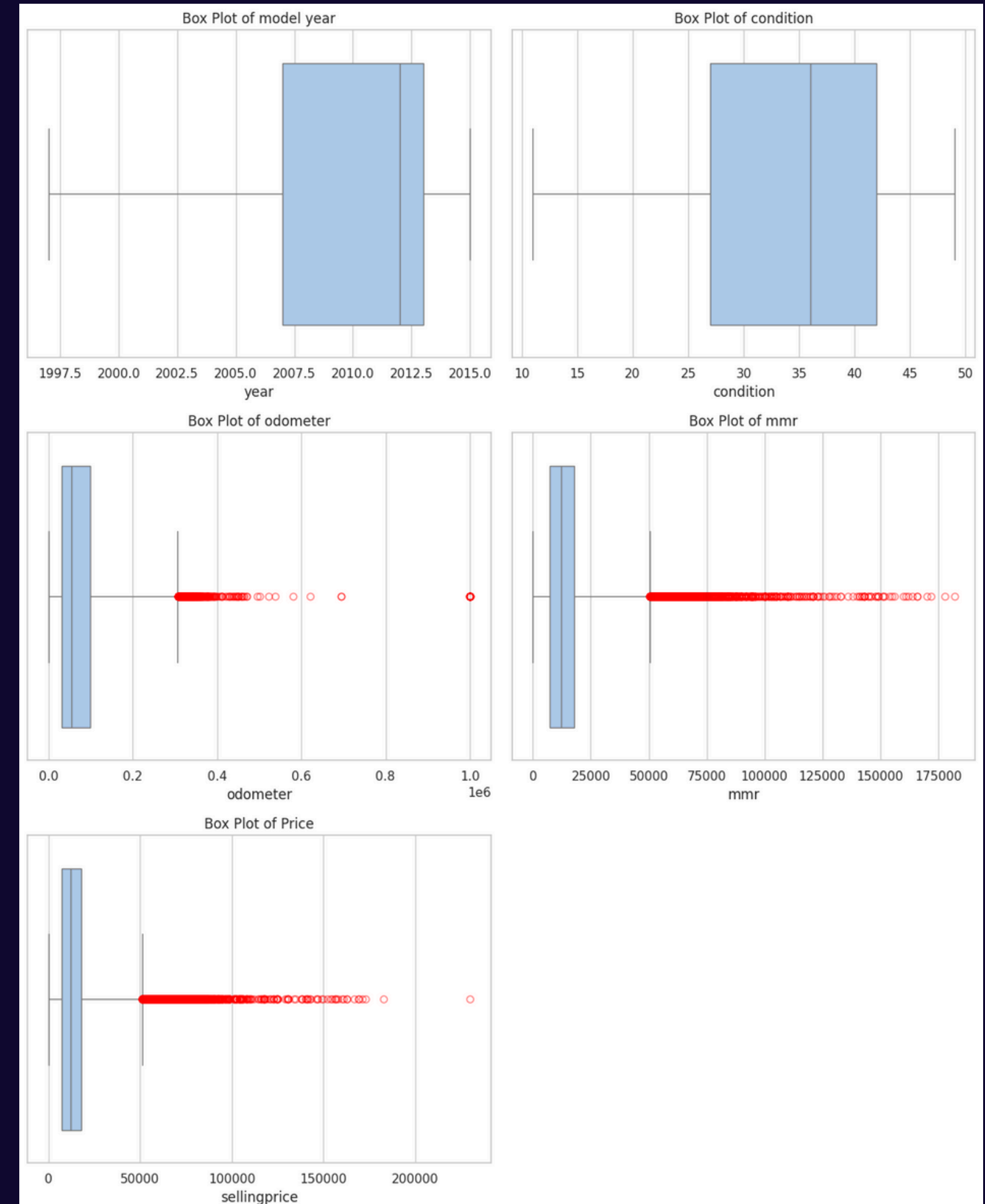
```
sns.boxplot(x=df['odometer'], ax=axes[1, 0], whis=3, flierprops=flierprops)
axes[1, 0].set_title('Box Plot of odometer')
```

```
sns.boxplot(x=df['mmr'], ax=axes[1, 1], whis=3, flierprops=flierprops)
axes[1, 1].set_title('Box Plot of mmr')
```

```
sns.boxplot(x=df['sellingprice'], ax=axes[2, 0], whis=3, flierprops=flierprops)
axes[2, 0].set_title('Box Plot of Price')
```

```
# Hide the empty subplot (last one, which has no plot)
fig.delaxes(axes[2, 1])
```

```
# Display the plots
plt.tight_layout()
plt.show()
```



```

▶ # Dictionary to hold the count of outliers for each column
outliers_count = {}

# Loop through each numerical column to calculate outliers
for column in numerical_columns:
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    # Define bounds for outliers
    lower_bound = Q1 - 3 * IQR
    upper_bound = Q3 + 3 * IQR

    # Count outliers
    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
    outliers_count[column] = len(outliers)

# Print the results
print("Outliers count per feature:")
for key, value in outliers_count.items():
    percent = round((value / len(df)) * 100, 2)
    print(f"{key}: {value} ({percent}%")

```

```

↔ Outliers count per feature:
year: 0 (0.0%)
condition: 0 (0.0%)
odometer: 417 (0.09%)
mmr: 2569 (0.55%)
sellingprice: 2387 (0.51%)

```

Delete the outliers

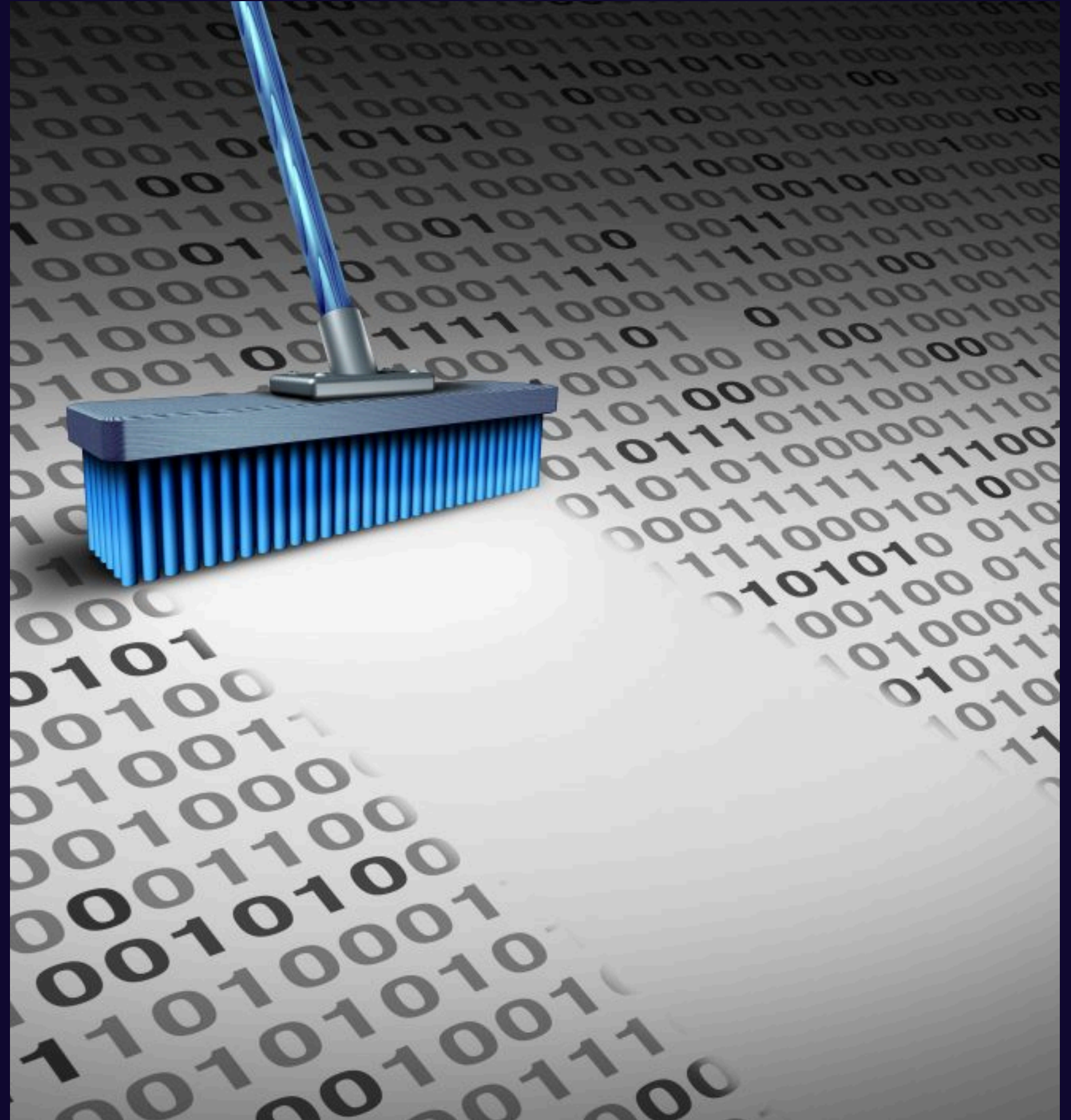
```

[ ] # Loop through each numerical column to calculate outliers
for column in numerical_columns:
    # Calculate the 1st and 3rd quartiles
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1

    # Define bounds for outliers (3 * IQR)
    lower_bound = Q1 - 3 * IQR
    upper_bound = Q3 + 3 * IQR

    # Filter out rows that fall outside the bounds
    df = df[(df[column] > lower_bound) & (df[column] < upper_bound)]

```



▼ One Hot Encoding

```
[ ] # Ensure that the 'saledate' column is of datetime type
    df['saledate'] = pd.to_datetime(df['saledate'])

    # Create a new column 'weekday' with values from 1 (Monday) to 7 (Sunday)
    df['weekday'] = df['saledate'].dt.weekday + 1
```

```
[ ] df1 = df.copy()
```

```
[ ] # Applying one-hot encoding to the columns 'body', 'make', and 'color'
    df1 = pd.get_dummies(df1, columns=["body"], prefix=["Cat"])
    df1 = pd.get_dummies(df1, columns=["make"], prefix=["Brand"])
    df1 = pd.get_dummies(df1, columns=["color"], prefix=["C"])
    df1 = pd.get_dummies(df1, columns=["interior"], prefix=["I"])

    # Converting the dummy columns to Int64 type
    dummy_columns = df1.columns[df1.columns.str.startswith('Cat_') | df1.columns.str.startswith('Brand_') | df1.columns.str.startswith('C') | df1.columns.str.startswith('I')]
    df1[dummy_columns] = df1[dummy_columns].astype('int64')
```

```
[ ] df1.shape
```

```
↔ (461705, 59)
```

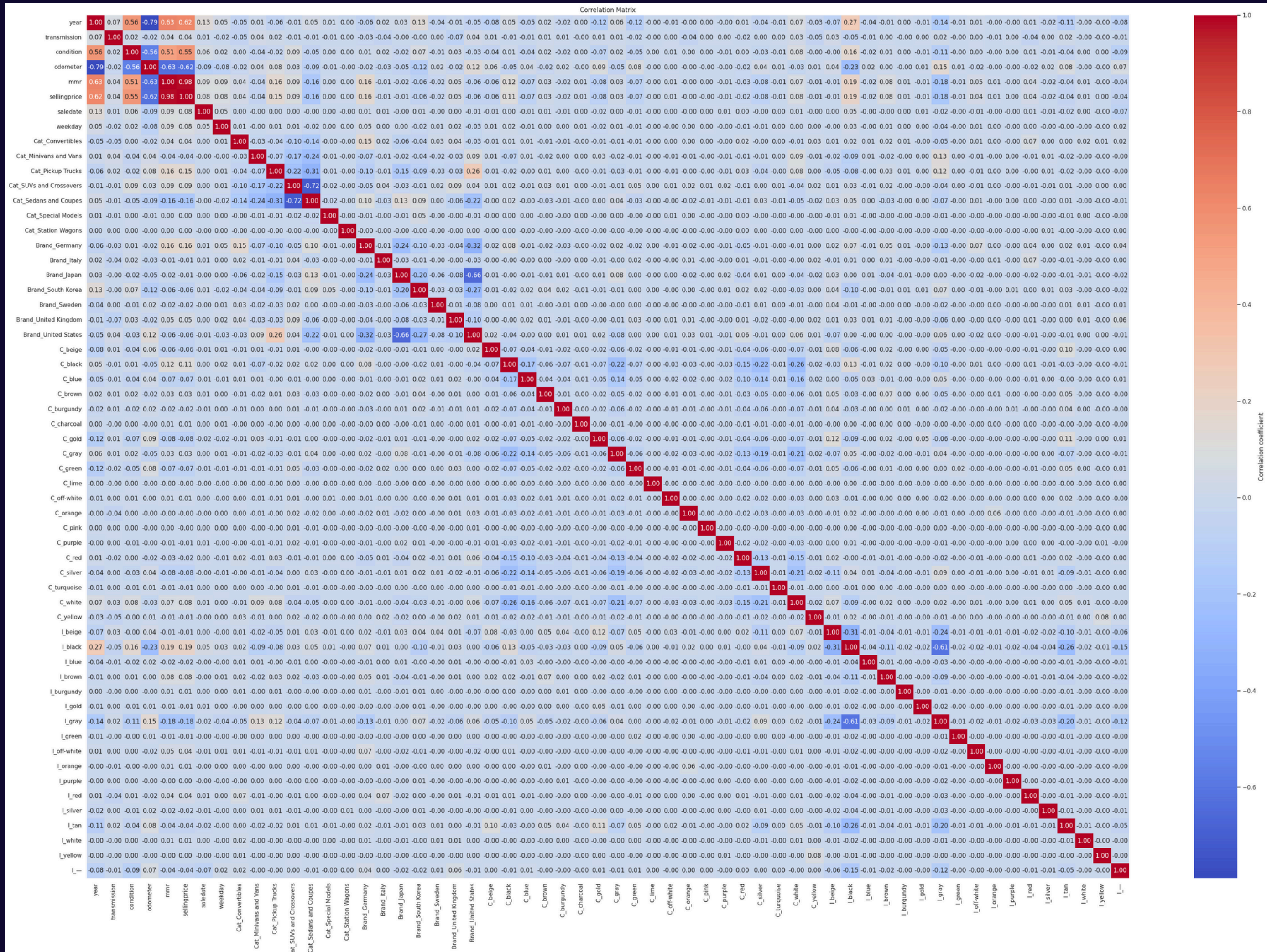
✓ Heat Map

```
[ ] from statsmodels.stats.outliers_influence import variance_inflation_factor
    from statsmodels.tools.tools import add_constant
```

```
[ ] # Select numerical columns from the DataFrame
    df_n = df1.select_dtypes(['float64', 'int64', 'int32', 'datetime64[ns]' ])

    # Calculate the correlation matrix
    corr_matrix = df_n.corr()

    # Plot the correlation matrix using a heatmap
    plt.figure(figsize=(45, 30)) # Set the size of the figure
    sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm',
                cbar_kws={'label': 'Correlation coefficient'})
    plt.title('Correlation Matrix')
    plt.show()
```

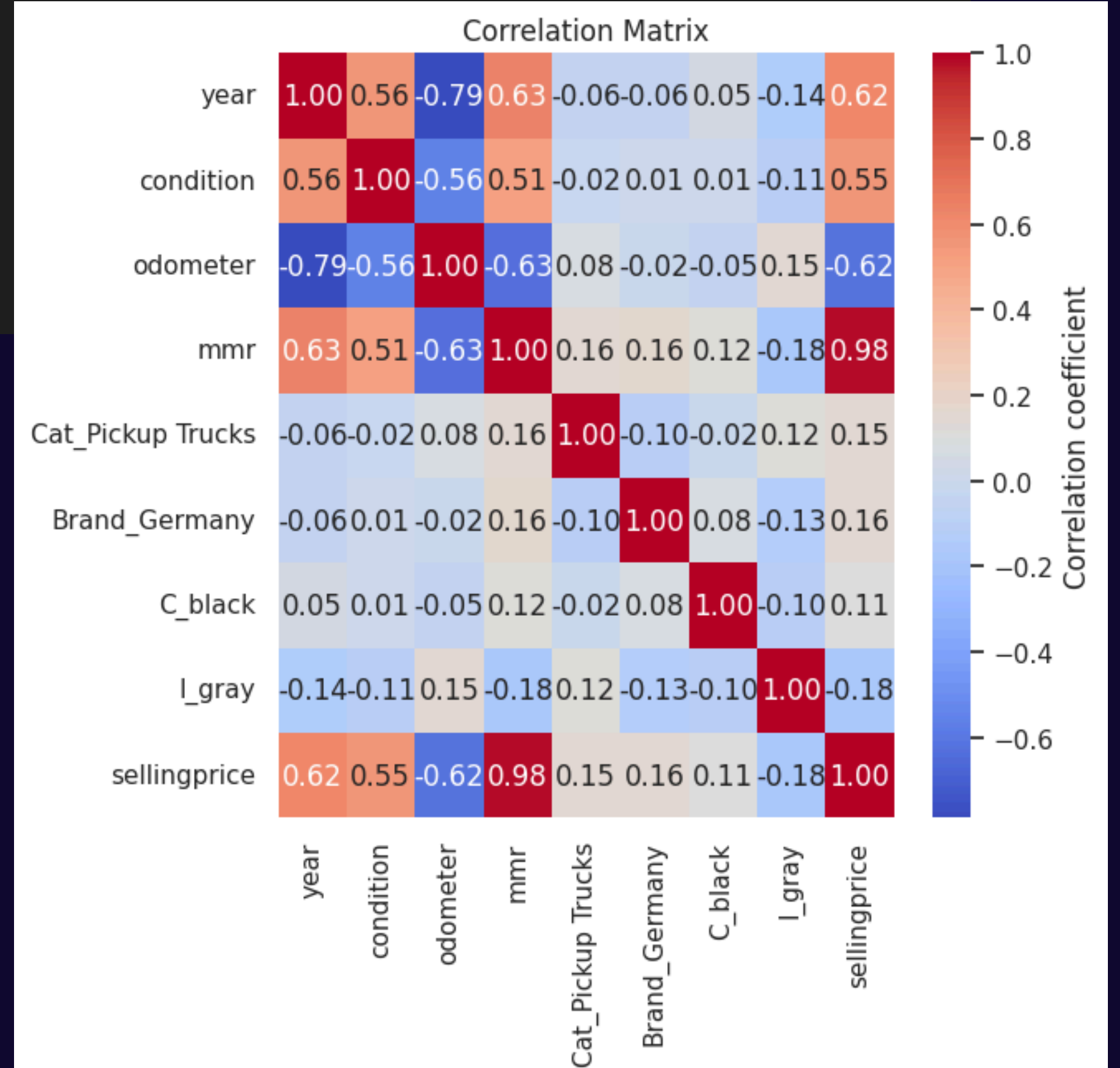


```
[ ] clean_df = df1[['year','condition','odometer','mmr','Cat_Pickup Trucks', 'Brand_Germany', 'C_black','I_gray', 'sellingprice']]
```

```
# Select numerical columns from the DataFrame
df_n = clean_df.select_dtypes(['float64', 'int64', 'int32'])

# Calculate the correlation matrix
corr_matrix = df_n.corr()

# Plot the correlation matrix using a heatmap
plt.figure(figsize=(6, 6)) # Set the size of the figure
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap='coolwarm',
            cbar_kws={'label': 'Correlation coefficient'})
plt.title('Correlation Matrix')
plt.show()
```



Predictive Models

✓ Lasso Regression

Before you run this model, please run step1(dataset selection) to step3(Data Cleaning and Preprocessing), because this training dataset is from there, it is not `clean_df`

```
▶ from sklearn.preprocessing import StandardScaler
  from sklearn import linear_model
  from sklearn.model_selection import train_test_split
  import numpy as np

  # Separate numeric and dummy variable features
  numeric_features = df1.select_dtypes(include=['int64', 'float64']).columns
  dummy_features = [col for col in numeric_features if df1[col].nunique() == 2] # Likely dummy variables
  non_dummy_numeric_features = numeric_features.difference(dummy_features)

  lasso_df = df1.copy()

  # Apply scaling to non-dummy numeric features
  scaler = StandardScaler()
  lasso_df[non_dummy_numeric_features] = scaler.fit_transform(lasso_df[non_dummy_numeric_features])
```

```
[ ] # Splitting the data into features and target
    X = lasso_df.drop(columns=['sellingprice', 'state', 'saledate'])
    y = lasso_df['sellingprice']

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # setting alpha = 0.01
    lasso2 = linear_model.Lasso(alpha=0.001, max_iter = 100000).fit(X_train, y_train)
    print("Training set score under Lasso regression: {:.4f}".format(lasso2.score(X_train, y_train)))
    print("Test set score under Lasso regression: {:.4f}".format(lasso2.score(X_test, y_test)))
    print("Number of features used under Lasso regression:", np.sum(lasso2.coef_ != 0))
```

```
⇒ Training set score under Lasso regression: 0.9727
   Test set score under Lasso regression: 0.9730
   Number of features used under Lasso regression: 9
```

Feature Scaling for Linear Regression Model

```
▶ from sklearn.preprocessing import StandardScaler

# Separate numeric and dummy variable features
numeric_features = clean_df.select_dtypes(include=['int64', 'float64']).columns
dummy_features = [col for col in numeric_features if clean_df[col].nunique() == 2] # Likely dummy variables
non_dummy_numeric_features = numeric_features.difference(dummy_features)

linear_df = clean_df.copy()

# Apply scaling to non-dummy numeric features
scaler = StandardScaler()
linear_df[non_dummy_numeric_features] = scaler.fit_transform(linear_df[non_dummy_numeric_features])
```

Linear Regression Model

```
# Splitting the data into features and target
Xl = linear_df.drop(columns=['sellingprice'])
yl = linear_df['sellingprice']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(Xl, yl, test_size=0.2, random_state=42)

# Creating and training the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

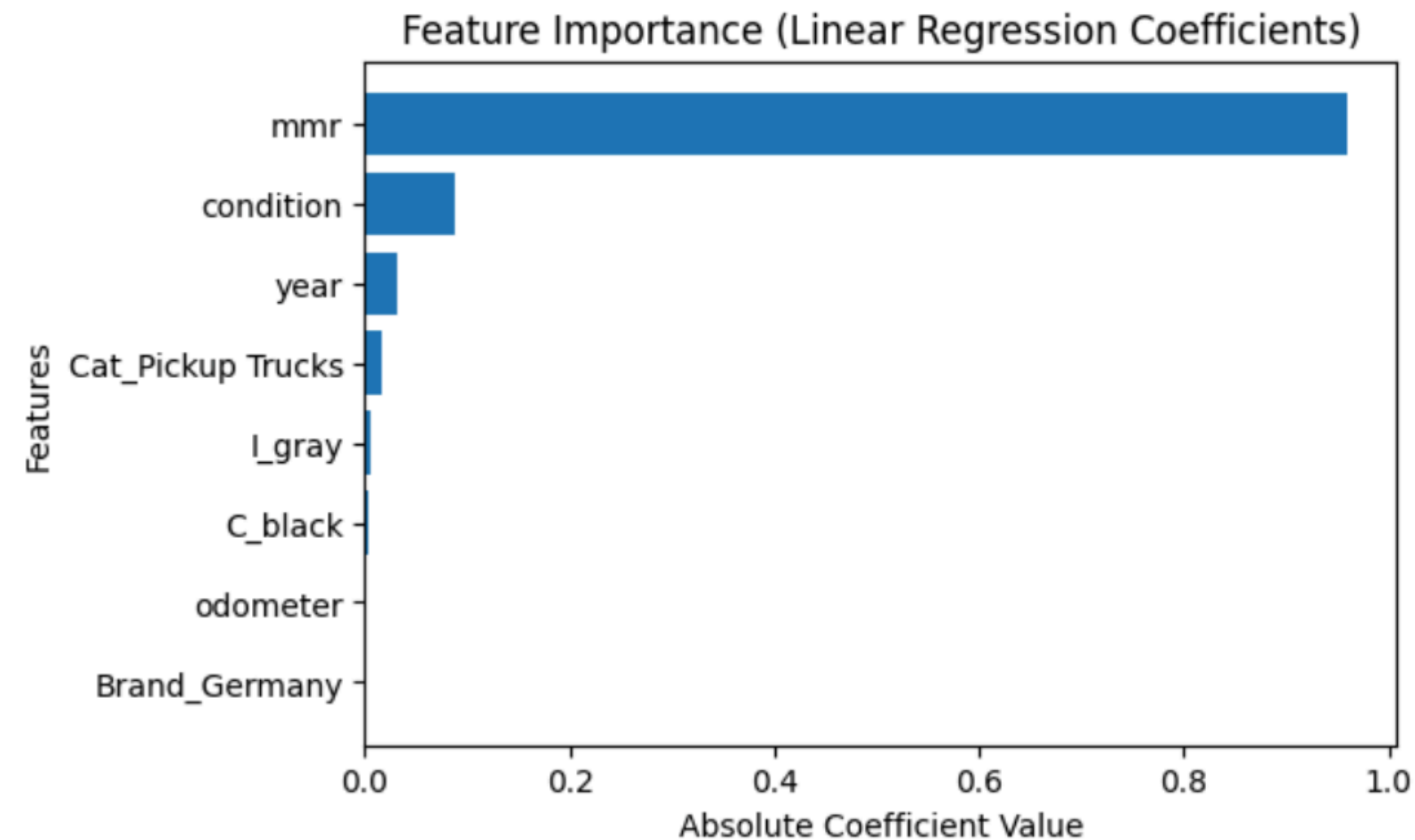
# Calculating metrics
mse = mean_squared_error(y_test, y_pred).round(4)
r2 = round(r2_score(y_test, y_pred), 4)
mae = mean_absolute_error(y_test, y_pred).round(4)
model_intercept = round(model.intercept_, 4)

# Printing model details for better readability
print("Model Intercept:", model_intercept)
print("Mean Squared Error (MSE):", mse)
print("R-squared (R2) Score:", r2)
print("Mean Absolute Error (MAE):", mae)
```

```
Model Intercept: -0.0004
Mean Squared Error (MSE): 0.0272
R-squared (R2) Score: 0.9729
Mean Absolute Error (MAE): 0.1093
```

Feature Importance DataFrame:

	Feature	Coefficient	Absolute Coefficient
3	mmr	0.960078	0.960078
1	condition	0.087131	0.087131
0	year	-0.032078	0.032078
4	Cat_Pickup Trucks	0.015654	0.015654
7	I_gray	-0.004939	0.004939
6	C_black	0.002381	0.002381
2	odometer	0.002211	0.002211
5	Brand_Germany	0.001845	0.001845



RandomForest Model

```
{'max_depth': 30, 'min_samples_leaf': 5, 'min_samples_split': 12, 'n_estimators': 100}
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# Adjusting the hyperparameters
rf2_model = RandomForestRegressor(n_estimators=100, max_depth=30,
                                min_samples_split=12, min_samples_leaf=5,
                                random_state=42)

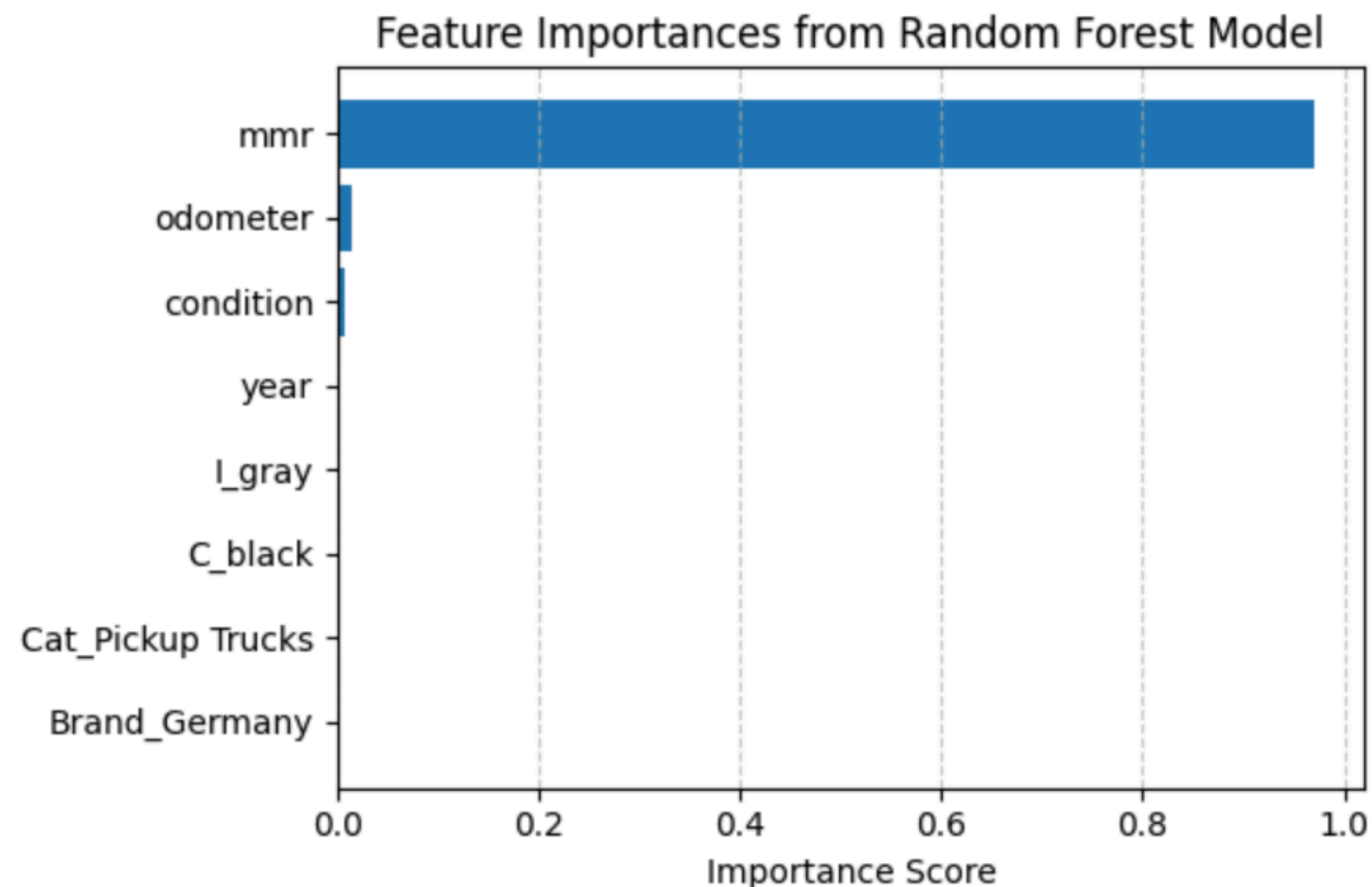
rf2_model.fit(X_train, y_train)

# Predictions using the adjusted RandomForest Regressor
y2_rf_pred = rf2_model.predict(X_test)

# Calculating metrics for RandomForest Regressor
rf2_mse = mean_squared_error(y_test, y2_rf_pred).round(4)
rf2_r2 = round(r2_score(y_test, y2_rf_pred), 4)
rf2_mae = mean_absolute_error(y_test, y2_rf_pred).round(4)

# Displaying MSE and R-squared value for RandomForest model
print("Mean Squared Error (MSE):", rf2_mse)
print("R-squared (R2) Score:", rf2_r2)
print("Mean Absolute Error (MAE):", rf2_mae)
```

```
Mean Squared Error (MSE): 0.0268
R-squared (R2) Score: 0.9733
Mean Absolute Error (MAE): 0.1086
```



✓ Gradient Boosting Model

```
▶ from sklearn.ensemble import GradientBoostingRegressor

# Configuring and training the Gradient Boosting model
gb_model = GradientBoostingRegressor(n_estimators=150, max_depth=5,
                                     min_samples_split=5, min_samples_leaf=3,
                                     random_state=42)

gb_model.fit(X_train, y_train)

# Predictions and evaluation
y_gb_pred = gb_model.predict(X_test)

# Calculating metrics for RandomForest Regressor
gb_mse = mean_squared_error(y_test, y_rf_pred).round(4)
gb_r2 = round(r2_score(y_test, y_rf_pred), 4)
gb_mae = mean_absolute_error(y_test, y_pred).round(4)

# Displaying MSE and R-squared value for RandomForest model
print("Mean Squared Error (MSE):", gb_mse)
print("R-squared (R2) Score:", gb_r2)
print("Mean Absolute Error (MAE):", gb_mae)
```

```
⇒ Mean Squared Error (MSE): 0.0285
   R-squared (R2) Score: 0.9716
   Mean Absolute Error (MAE): 0.1093
```

Business Questions

Business Question 1: What are the key factors that influence the selling price of used vehicles?

Key Insights:

MMR Value: The most significant predictor of selling price. Vehicles sold close to their MMR value, emphasizing its reliability as a pricing benchmark.

- Action: Use MMR as a reference; price slightly above for high demand or unique features, and closer to MMR for slower-moving inventory.

Vehicle Condition: Higher condition scores lead to 10–15% higher selling prices.

- Action: Invest in minor repairs and refurbishments to boost condition ratings and perceived value.

Age and Depreciation: Steep depreciation in the first five years; 3–7-year-old vehicles offer the best balance of value and affordability.

- Action: Stock 3–7-year-old vehicles and highlight affordability to attract buyers.

Body Type Preferences: SUVs and pickup trucks fetch higher prices; sedans dominate volume sales.

- Action: Tailor inventory to regional preferences and promote key features like towing capacity for trucks.

Color Preferences: Neutral colors (black, silver, white) sell faster and at higher prices, while rare colors have niche appeal.

- Action: Focus inventory on high-demand colors and use targeted campaigns for rare ones.

Business Question 2: How can we accurately predict the selling price of a vehicle based on its features?

Key Insights:

- The Random Forest Regressor achieved a high R^2 score (0.973), indicating it is a robust predictive model for estimating selling prices.
- The model incorporates both linear and non-linear relationships among variables, making it suitable for the dataset's complexity.
- Feature importance analysis reveals that `MMR`, `year` and `condition` are the most influential predictors.

Actionable Insight:

- Deploy the Random Forest model in dealership operations to provide accurate price estimates for trade-ins and sales negotiations.
- Simplify the model by focusing on the most important features (e.g., `MMR`, `condition`, and `year`) for operational ease without significantly sacrificing accuracy.

Business Question 3: What trends and patterns exist in vehicle sales over time, and how do these trends vary across different segments?

Key Insights:

- Economic Trends: A noticeable dip in the availability of 2008–2009 models reflects the impact of the 2008 financial crisis, suggesting economic events significantly influence vehicle sales.
- Transmission Type: Automatic transmission dominates the market (~96%), indicating strong consumer preference.
- Color and Interior Trends: Black exteriors and gray interiors are more popular and slightly increase perceived vehicle value.

Actionable Insight:

- Monitor broader economic conditions and their potential impacts on vehicle demand and pricing trends.
- Stock vehicles with automatic transmissions, as they align with dominant consumer preferences.
- Target marketing campaigns toward popular vehicle colors and interior styles, emphasizing these features to attract buyers.

Summary of Expected Outcomes:

- **Enhanced Pricing Strategies:** By leveraging the predictive model, the dealership can set competitive yet profitable prices, reducing the risk of overpricing or underpricing vehicles.
- **Streamlined Inventory Management:** Prioritize stocking vehicles that align with consumer demand, such as pickup trucks, automatic transmissions, and popular color combinations.
- **Data-Driven Decision Making:** Incorporating insights from economic trends, vehicle features, and consumer preferences ensures a strategic approach to sales and marketing.

THE END